

Hành trình Điều tra Nguyên nhân Gốc rễ (Root Cause Investigation Journey)

Mở đầu: Một bức tranh hỗn độn

Khi mới tiếp nhận bộ dữ liệu, chúng tôi đối mặt với hàng trăm ngàn dòng log sự kiện. Không có manh mối rõ ràng nào. Chúng tôi bắt đầu bằng một câu hỏi đơn giản: **"Liệu tất cả các máy móc trong nhà máy có đang gặp cùng một vấn đề không?"**

Để trả lời, chúng tôi quyết định chia nhà máy thành các "gia đình" máy (Machine Family) dựa trên mã dây chuyền của chúng.

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import re

sns.set_style("whitegrid")
df = pd.read_csv('master_data.csv', low_memory=False)
df['Total_Production'] = df['GOOD_PRODUCTION_QTY'] + df['REJECT_PRODUCTION_QTY']

# --- BƯỚC 1: PHÂN LOẠI DÒNG MÁY (MACHINE FAMILY) ---
# Chúng tôi nhận thấy các mã LINE_NAME có tiền tố giống nhau (MAFT, MKBC...)
def get_machine_family(line_name):
    if pd.isna(line_name): return 'Other'
    if str(line_name).startswith('MAFT'): return 'MAFT'
    if str(line_name).startswith('MKBC'): return 'MKBC'
    if str(line_name).startswith('MKTU'): return 'MKTU'
    if str(line_name).startswith('MHPT'): return 'MHPT'
    return 'Other'

df['Machine_Family'] = df['LINE_NAME'].apply(get_machine_family)

# --- BƯỚC 2: QUAN SÁT TỔNG QUAN ---
# Tính toán các chỉ số KPI cơ bản cho từng dòng máy để tìm sự bất thường
family_overview = df.groupby('Machine_Family').agg(
    Total_Downtime=('DOWNTIME', 'sum'),
    Total_Defect=('REJECT_PRODUCTION_QTY', 'sum'),
    Total_Production=('Total_Production', 'sum')
).reset_index()
family_overview['Defect_Rate_%'] = (family_overview['Total_Defect'] / family_overview['Total_Production']) * 100
family_overview['Downtime_Share_%'] = (family_overview['Total_Downtime'] / family_overview['Total_Production']) * 100

print("BẢNG 1: TỔNG QUAN HIỆU SUẤT THEO DÒNG MÁY")
display(family_overview.sort_values('Total_Downtime', ascending=False))
```

BẢNG 1: TỔNG QUAN HIỆU SUẤT THEO DÒNG MÁY

	Machine_Family	Total_Downtime	Total_Defect	Total_Production	Defect_Rate_%	Dowr
2	MKBC	518527	3690614	268101296	1.38	
4	Other	381247	1444231	80435821	1.80	
1	MHPT	153395	1110507	76094354	1.46	
3	MKTU	122802	617938	19156650	3.23	
0	MAFT	95837	368732	7353563	5.01	



Manh mối đầu tiên: Sự đối lập kỳ lạ

Bảng số liệu trên ngay lập tức cho thấy một sự đối lập đáng ngờ:

- **MAFT**: Có **Defect Rate cao nhất** (5.01%) nhưng Downtime lại khá thấp (chỉ 7.5%).
- **MKBC**: Có **Downtime cao nhì** (40.8%) nhưng Defect Rate lại rất thấp (1.38%).

Giả thuyết mới: Nhà máy đang có HAI "căn bệnh" riêng biệt. Chúng tôi quyết định tách đôi cuộc điều tra: Đội 1 điều tra Downtime trên MKBC, Đội 2 điều tra Chất lượng trên MAFT.

Hướng điều tra 1: Truy tìm "kẻ thủ ác" gây Downtime trên MKBC

Chúng tôi bắt đầu "đào bới" cột `UTIL_REASON_DESCRIPTION` của riêng dòng MKBC.

Dữ liệu ở đây là dạng văn bản tự do, rất hỗn loạn. Chúng tôi cần tìm ra các từ khóa xuất hiện nhiều nhất.

```
In [2]: # Lọc dữ liệu chỉ cho MKBC
mkbc_df = df[df['Machine_Family'] == 'MKBC'].copy()

# --- BƯỚC 3: KHÁM PHÁ TỪ KHÓA DOWNTIME (TEXT MINING ĐƠN GIẢN) ---
# Chúng tôi đếm tần suất xuất hiện của các mô tả lý do để tìm manh mối
downtime_reasons = mkbc_df[mkbc_df['DOWNTIME'] > 0]['UTIL_REASON_DESCRIPTION'].v

print("TOP 20 LÝ DO DOWNTIME GHI NHẬN TRÊN MKBC:")
print(downtime_reasons)

# --- QUAN SÁT ---
# Chúng tôi thấy các cụm từ như "Jammed handle", "Double handle", "Stuck handle"
# -> TỪ KHÓA (KEY) ĐƯỢC TÌM THẤY: "HANDLE" và các biến thể lỗi của nó (jam, stuc
```

TOP 20 LÝ DO DOWNTIME GHI NHẬN TRÊN MKBC:

UTIL_REASON_DESCRIPTION	
17-End Feeder - 1702-Jammed handle on chain	3536
13-FM chain - 1306-Handle stuck on FM	3025
17-End Feeder - 1701-Double handle on chain	1862
11-Auto handle feederT1-1101-Double handle on chain	1645
11-Auto handle feederT2-1101-Double handle on chain	1368
11-Auto handle feederT3-1101-Double handle on chain	1319
11-Auto handle feederT2-1102-Stuck handle on chain	1219
11-Auto handle feederT1-1102-Stuck handle on chain	1118
11-Auto handle feederT3-1102-Stuck handle on chain	1058
02-Quality - 0202-Rework	656
14-Fiber picker T2-1401-Adjust gap of fiber picker and picker guide	545
17-End Feeder - 1705-Mechanical Adjustment	497
14-Fiber picker T1-1401-Adjust gap of fiber picker and picker guide	410
11-Auto handle feederT3-1103-Adjust robot position	398
17-End Feeder - 1712-Lack of handle on chain End Feeder	365
14-Fiber picker T3-1401-Adjust gap of fiber picker and picker guide	322
11-Auto handle feederT1-1104-Lack of handle on chain	318
06-Fiber boxT2-0602-Replace/adjust mechanical parts of fiber beaters	269
11-Auto handle feederT3-1104-Lack of handle on chain	262
11-Auto handle feederT1-1103-Adjust robot position	258

Name: count, dtype: int64



Manh mối thứ hai: "Handle Jam" là nghi phạm chính

Từ việc quan sát top 20 lý do, chúng tôi thấy một mẫu hình rõ ràng: rất nhiều lỗi liên quan đến việc cấp tay cầm (handle). Chúng tôi quyết định gom tất cả các lỗi này lại thành một nhóm gọi là "**Handle Jam**" để xem tổng tác động của nó.

```
In [3]: # --- BƯỚC 4: KIỂM CHỨNG GIẢ THUYẾT "HANDLE JAM" ---
# Tạo một hàm để gom nhóm các Lỗi Liên quan đến Handle Jam
def is_handle_jam(reason):
    if pd.isna(reason): return False
    r = str(reason).lower()
    # Logic: Nếu chứa từ "handle" VÀ chứa một trong các từ chỉ lỗi ("jam", "stuc
    return 'handle' in r and ('jam' in r or 'stuck' in r or 'double' in r)

mkbc_df['Is_Handle_Jam'] = mkbc_df['UTIL_REASON_DESCRIPTION'].apply(is_handle_ja

# Tính tổng tác động
total_downtime = mkbc_df['DOWNTIME'].sum()
handle_jam_downtime = mkbc_df[mkbc_df['Is_Handle_Jam']][ 'DOWNTIME'].sum()
jam_percentage = (handle_jam_downtime / total_downtime * 100)

print(f"TỔNG KẾT ĐIỀU TRA TRÊN MKBC:")
print(f"- Tổng Downtime: {total_downtime:,.0f} phút")
print(f"- Downtime do 'Handle Jam': {handle_jam_downtime:,.0f} phút")
print(f"- Tỷ trọng: {jam_percentage:.1f}%")

# -> KẾT LUẬN 1: "Handle Jam" chính là nguyên nhân gốc gây ra gần một nửa số dow
```

TỔNG KẾT ĐIỀU TRA TRÊN MKBC:

- Tổng Downtime: 518,527 phút
- Downtime do 'Handle Jam': 242,440 phút
- Tỷ trọng: 46.8%

Hướng điều tra 2: Truy tìm nguyên nhân Defect cao trên MAFT

Quay lại với dòng MAFT. Tại sao defect rate lại cao đột biến (5.01%)? Chúng tôi đặt ra các giả thuyết:

1. **Do máy hỏng?** (Kiểm tra từng máy trong dòng MAFT)
2. **Do con người?** (Kiểm tra từng ca/đội)
3. **Do sản phẩm khó?** (Kiểm tra từng loại sản phẩm)

```
In [4]: # Lọc dữ liệu sản xuất chỉ cho MAFT
maft_prod = df[(df['Machine_Family'] == 'MAFT') & (df['Total_Production'] > 0)].
maft_prod['Defect_Rate'] = maft_prod['REJECT_PRODUCTION_QTY'] / maft_prod['Total

# --- BƯỚC 5: RÀ SOÁT CÁC "NGHI PHẠM" (MÁY, CA, SẢN PHẨM) ---

# Kiểm tra theo Sản phẩm (SIZE_TYPE)
product_check = maft_prod.groupby('SIZE_TYPE').agg(
    Avg_Defect_Rate=('Defect_Rate', 'mean'),
    Total_Production=('Total_Production', 'sum')
).sort_values('Avg_Defect_Rate', ascending=False).head(5)

print("TOP 5 SẢN PHẨM CÓ DEFECT RATE CAO NHẤT TRÊN MAFT:")
display(product_check)

# -> QUAN SÁT: "Renew 360" đứng đầu bảng với defect rate cao và sản lượng rất LỚ
# -> TỪ KHÓA (KEY) ĐƯỢC TÌM THẤY: "Renew 360".
```

TOP 5 SẢN PHẨM CÓ DEFECT RATE CAO NHẤT TRÊN MAFT:

	Avg_Defect_Rate	Total_Production
SIZE_TYPE		
Renew 360	0.054281	6055265
Oral Care 360	0.045834	1298298



Manh mối thứ ba: "Renew 360" là nghi phạm chính

Sản phẩm "Renew 360" có vẻ đáng ngờ. Chúng tôi cần kiểm chứng xem liệu nó có thực sự khác biệt so với các sản phẩm còn lại không, hay đây chỉ là sự ngẫu nhiên.

```
In [5]: # --- BƯỚC 6: KIỂM CHỨNG GIẢ THUYẾT "RENEW 360" (SO SÁNH ĐỐI CHỨNG) ---
maft_prod['Product_Group'] = np.where(maft_prod['SIZE_TYPE'] == 'Renew 360', 'Re

comparison = maft_prod.groupby('Product_Group').agg(
    Total_Defect=('REJECT_PRODUCTION_QTY', 'sum'),
    Total_Production=('Total_Production', 'sum')
).reset_index()
comparison['Defect_Rate_%'] = (comparison['Total_Defect'] / comparison['Total_Pr

print("BẢNG SO SÁNH: RENEW 360 vs. CÁC SẢN PHẨM KHÁC (TRÊN CÙNG DÒNG MAFT)")
display(comparison)

# --- BƯỚC 7: KIỂM ĐỊNH THỐNG KÊ (ĐỂ CHẮC CHẮN 100%) ---
```

```
from scipy.stats import ttest_ind
group1 = maft_prod[maft_prod['Product_Group'] == 'Renew 360']['Defect_Rate']
group2 = maft_prod[maft_prod['Product_Group'] == 'Others']['Defect_Rate']
t_stat, p_val = ttest_ind(group1, group2, equal_var=False)

print(f"Kết quả kiểm định t-test: P-value = {p_val:.5f}")
if p_val < 0.05:
    print("-> KẾT LUẬN: Sự khác biệt là CÓ Ý NGHĨA THỐNG KÊ. 'Renew 360' thực sự
else:
    print("-> KẾT LUẬN: Không đủ bằng chứng để khẳng định sự khác biệt.")
```

BẢNG SO SÁNH: RENEW 360 vs. CÁC SẢN PHẨM KHÁC (TRÊN CÙNG DÒNG MAFT)

	Product_Group	Total_Defect	Total_Production	Defect_Rate_%
0	Others	58947	1298298	4.54
1	Renew 360	309785	6055265	5.12

Kết quả kiểm định t-test: P-value = 0.00078

-> KẾT LUẬN: Sự khác biệt là CÓ Ý NGHĨA THỐNG KÊ. 'Renew 360' thực sự khó sản xuất hơn.

Tổng kết Điều tra: Vụ án đã được phá

Qua quá trình điều tra từng bước, từ quan sát tổng quan, phát hiện manh mối, đến kiểm chứng giả thuyết bằng thống kê, chúng tôi đã đi đến kết luận cuối cùng:

- Thủ phạm gây Downtime trên MKBC:** Là lỗi cơ khí hệ thống, cụ thể là **"Handle Jam"** (chiếm 46.8% downtime).
- Thủ phạm gây Defect trên MAFT:** Là sản phẩm mới **"Renew 360"**, vốn có độ khó sản xuất cao hơn đáng kể so với các sản phẩm khác.

Chúng tôi không còn dựa vào phỏng đoán. Mọi kết luận đều được xây dựng từ bằng chứng dữ liệu vững chắc.